Methods Review

MY459 Computational Text Analysis and Large Language Models

https://lse-my459.github.io/

Friedrich Geiecke

Remaining weeks

1. Methods review
2. Introduction to neural networks
3. Fundamentals of word embeddings
4. Large language models

Today

- Before we begin with neural networks, it is very helpful to review some selected topics in linear algebra, calculus, and optimisation

Outline

- Vectors & matrices
- Derivatives
- Gradient descent

Vectors

- Vectors such as $a = \begin{pmatrix} 7 \\ -3 \\ 5.3 \\ -2.7 \end{pmatrix}$ can be interpreted in different ways, e.g. as ordered lists storing numbers, points in space, or representations of direction
- Operations with vectors and matrices, or more generally tensors, underlie much of deep learning
- Visual intuition is very helpful, e.g. for the discussion of embeddings in the next weeks

Vector spaces

- A vector space is a set V of vectors with two fundamental operations: **Scalar multiplication** and **vector addition**
- Applied to the vector spaces with real numbers $\mathbb{R}^n$ that we typically think about in this course:
  - **Scalar multiplication:** Scale a vector by a real number. For any scalar $\alpha \in \mathbb{R}$ and vector $v = (v_1, v_2, \ldots, v_n)$,

$$\alpha v = (\alpha v_1,\ \alpha v_2,\ \ldots,\ \alpha v_n)$$

  - **Vector addition:** Combine two vectors to produce another vector in the same space. For $u = (u_1, u_2, \ldots, u_n)$ and $v = (v_1, v_2, \ldots, v_n)$ in $\mathbb{R}^n$, the sum is given by

$$u + v = (u_1 + v_1,\ u_2 + v_2,\ \ldots,\ u_n + v_n)$$

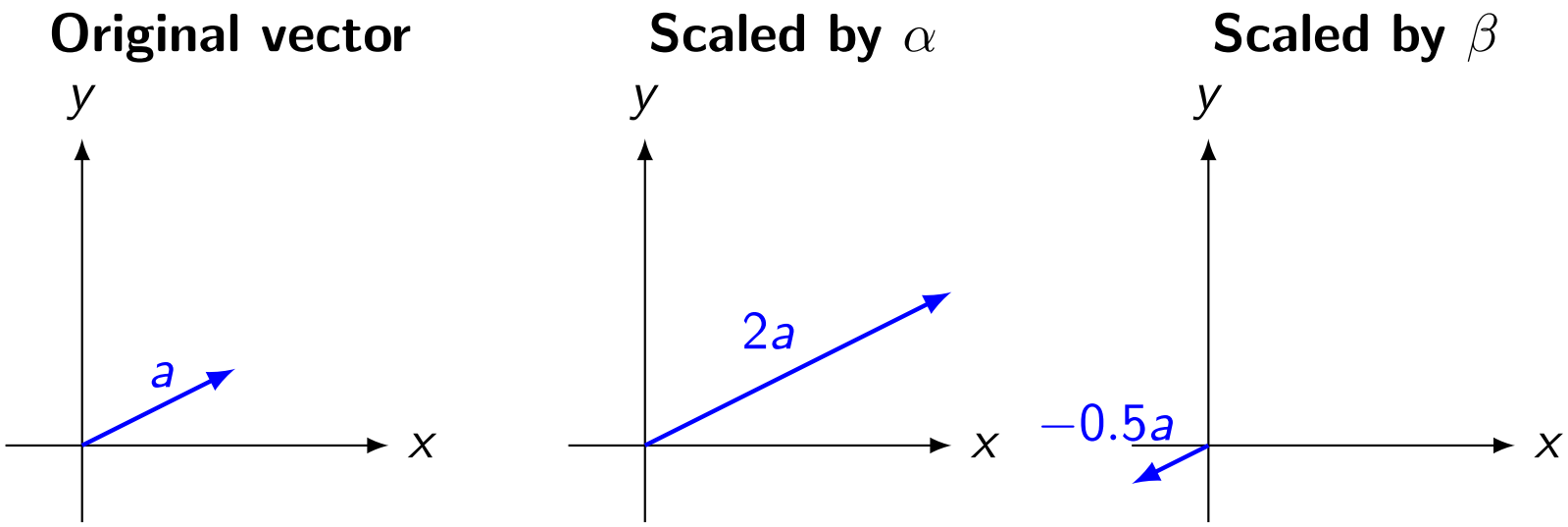
Visual intuition: Scalar multiplication

Consider a vector

$$a = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

multiplied by scalars $\alpha = 2$ and $\beta = -0.5$:

$$2a = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad -0.5a = \begin{pmatrix} -1 \\ -0.5 \end{pmatrix}$$

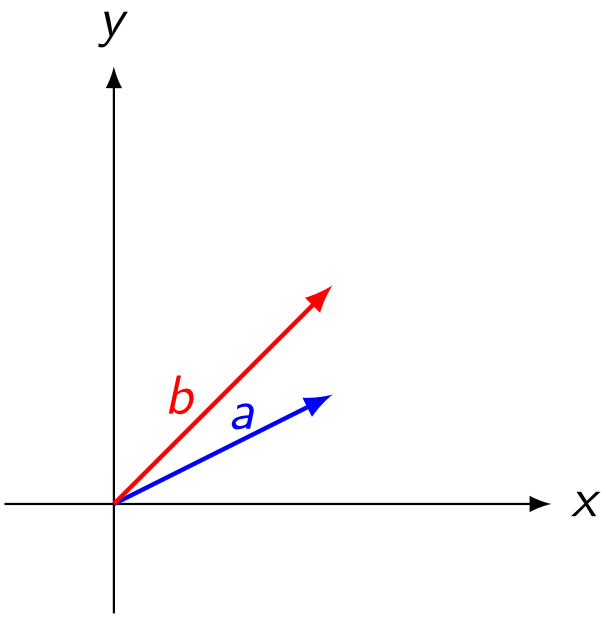


Visual intuition: Vector addition

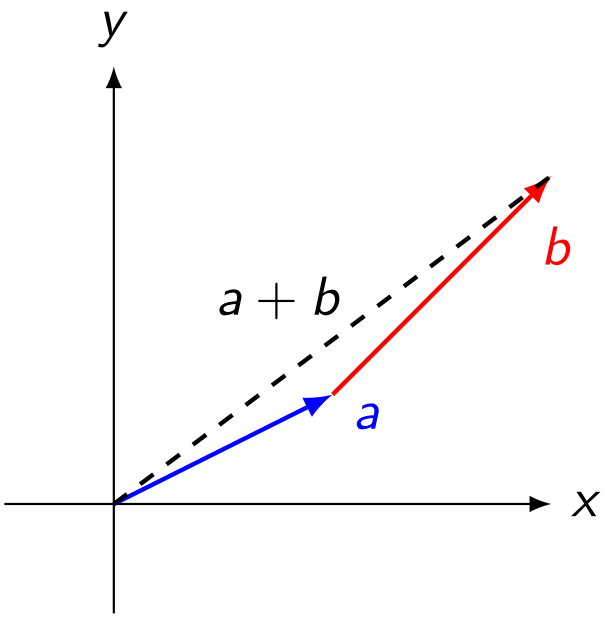
$$a = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad b = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

$$a + b = \begin{pmatrix} 2+2 \\ 1+2 \end{pmatrix} = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$$

Individual vectors



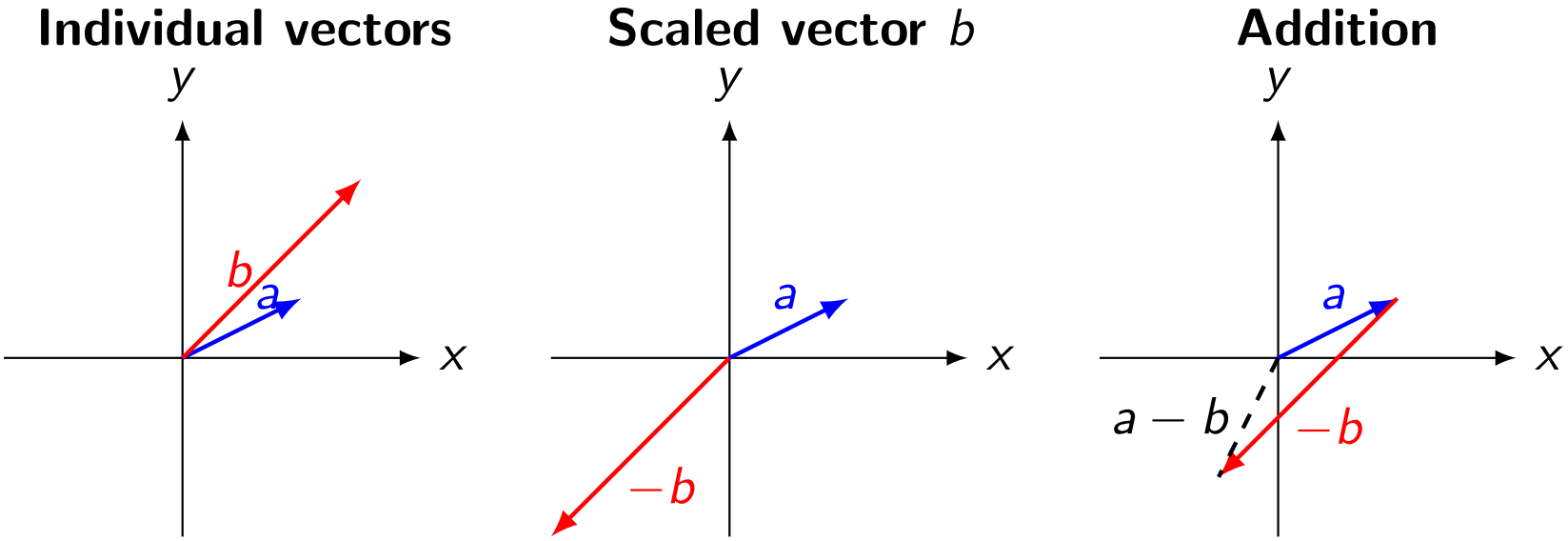
Addition



Visual intuition: Vector subtraction

$$a = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 3 \end{pmatrix},$$

$$a - b = a + (-b) = \begin{pmatrix} 2-3 \\ 1-3 \end{pmatrix} = \begin{pmatrix} -1 \\ -2 \end{pmatrix}$$



The dot product

- **Definition:** For vectors $u, v \in \mathbb{R}^n$, the dot product is defined as

$$u \cdot v = \sum_{i=1}^{n} u_i v_i$$

- It produces a scalar. For example, for $u = (1, 3, -2)$ and $v = (0, 5, -2)$:

$$u \cdot v = 1 \times 0 + 3 \times 5 + -2 \times -2 = 19$$

Matrix multiplication

- Matrices are rectangular arrays with rows and columns
- Matrix multiplication generalises the dot product between vectors
- It is a key operation in neural networks and standard regression models
- Properties of matrix multiplication:
  - Compatibility: The number of columns in A must equal the number of rows in B
  - Associativity: $(AB)C = A(BC)$
  - Distributivity: $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$
  - Identity: $AI = IA = A$, where $I_n = \mathrm{diag}(1, 1, \ldots, 1)$
  - Scalar multiplication: $c(AB) = (cA)B = A(cB)$ for any scalar c
  - Non-commutativity: In general, $AB \neq BA$

Definition and examples

Let $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$. Then the product $AB \in \mathbb{R}^{m\times p}$ is given by

$$(AB)_{ik} = \sum_{j=1}^{n} a_{ij} b_{jk}, \quad \text{for } i = 1, \ldots, m \text{ and } k = 1, \ldots, p.$$

Example 1:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \text{and} \quad x = \begin{pmatrix} 5 \\ 6 \end{pmatrix}$$

$$\underbrace{A}_{2\times 2}\underbrace{x}_{2\times 1} = \begin{pmatrix} 1\cdot 5 + 2\cdot 6 \\ 3\cdot 5 + 4\cdot 6 \end{pmatrix} = \begin{pmatrix} 5+12 \\ 15+24 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}$$

Example 2:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \end{pmatrix}$$

$$\underbrace{A}_{2\times 2}\underbrace{B}_{2\times 2} = \begin{pmatrix} 1\cdot 7 + 2\cdot 9 & 1\cdot 8 + 2\cdot 10 \\ 3\cdot 7 + 4\cdot 9 & 3\cdot 8 + 4\cdot 10 \end{pmatrix} = \begin{pmatrix} 7+18 & 8+20 \\ 21+36 & 24+40 \end{pmatrix} = \begin{pmatrix} 25 & 28 \\ 57 & 64 \end{pmatrix}$$

Example 3:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \end{pmatrix}.$$

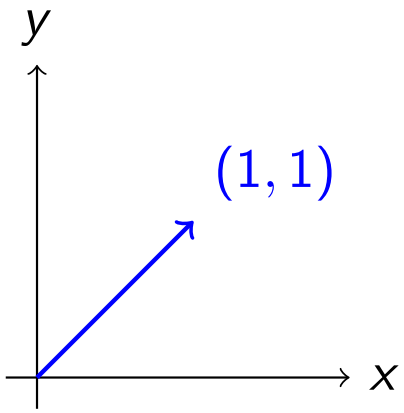
$$\underbrace{A}_{3\times 2}\underbrace{B}_{2\times 2} = \begin{pmatrix} 1\cdot 7 + 2\cdot 9 & 1\cdot 8 + 2\cdot 10 \\ 3\cdot 7 + 4\cdot 9 & 3\cdot 8 + 4\cdot 10 \\ 5\cdot 7 + 6\cdot 9 & 5\cdot 8 + 6\cdot 10 \end{pmatrix} = \begin{pmatrix} 25 & 28 \\ 57 & 64 \\ 89 & 100 \end{pmatrix}.$$

Visual intuition: Matrix multiplication

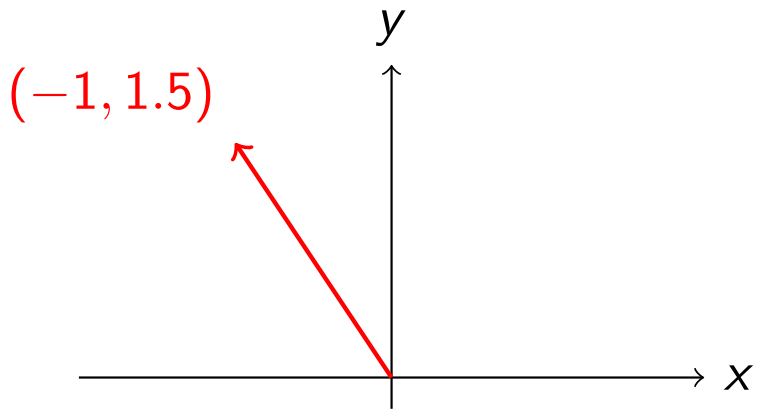
- While often taught primarily as an algorithm, matrix multiplication has a nice geometric interpretation. It linearly moves ("transforms") points in a space to new positions in the same or another space

- For example, multiplying vectors with $A = \begin{pmatrix} 1 & -2 \\ -0.5 & 2 \end{pmatrix}$:

Exemplary vector $(1, 1)$



New position $(-1, 1.5)$



$$A \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + (-2) \cdot 1 \\ -0.5 \cdot 1 + 2 \cdot 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 1.5 \end{pmatrix}$$

Inverting matrices

- Exercise in the next class

Cosine similarity

- Lastly, let us briefly review cosine similarity between vectors which we use e.g. to compute similarities between embeddings that encode documents, words, etc.

$$\text{cosine similarity}(a, b) = \frac{a \cdot b}{\|a\|\|b\|} \in [-1, 1]$$

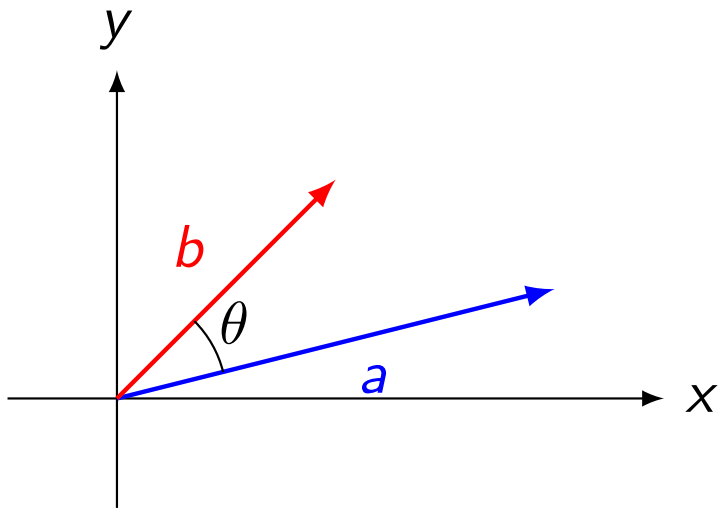
- The length of a vector is given by $\|x\| = \sqrt{x \cdot x} = \sqrt{\sum_{i=1}^{n} x_i^2}$
- Note that cosine similarity follows directly from the geometric interpretation of the dot product:

$$u \cdot v = \|u\|\|v\| \cos\theta$$

where θ is the angle between u and v

Visual intuition: Cosine similarity

$$a = \begin{pmatrix} 4 \\ 1 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad \frac{a \cdot b}{\|a\|\|b\|} \approx 0.8575 = cos(\theta)$$



In radians: $cos^{-1}(0.8575) = \theta \approx 0.5404$

In degrees: $\theta_{degrees} = 0.5404 \frac{180}{\pi} \approx 30.96^\circ$

Outline

- Vectors & matrices
- Derivatives
- Gradient descent

Calculus

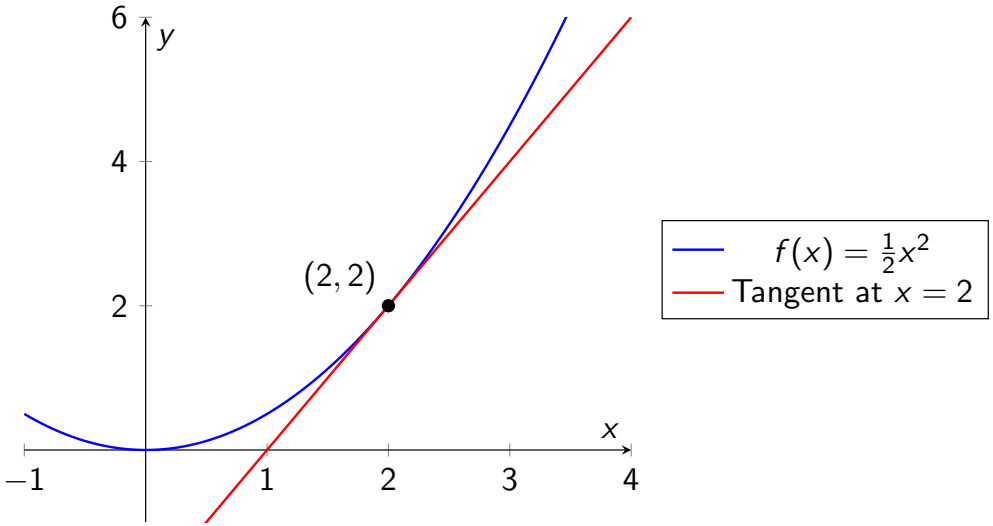
- Next, we will review some simple derivatives of univariate functions, partial derivatives of multivariate functions, gradient vectors, and the chain rule
- Even training current neural networks often just requires sequences of first-order derivatives via the chain rule
- All generic functions in the following stylised overview are assumed to be differentiable

Definition

Definition (limit form):

$$\frac{df}{dx} \equiv f'(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

- $f'(x)$ is the rate of change of f at the point x
- Graphically, $f'(x)$ is the slope of the tangent line to $f(x)$ at point x



Examples

Some common cases:

$$\frac{d}{dx}(x^n) = nx^{n-1}, \quad \frac{d}{dx}(e^x) = e^x, \quad \frac{d}{dx}(\log x) = \frac{1}{x}.$$

- Linearity: $\frac{d}{dx}\big(a \cdot f(x) + b \cdot g(x)\big) = a \cdot f'(x) + b \cdot g'(x)$.
- Product Rule: $\frac{d}{dx}\big(f(x)g(x)\big) = f'(x)g(x) + f(x)g'(x)$.
- Quotient Rule: $\frac{d}{dx}\left(\frac{f(x)}{g(x)}\right) = \frac{f'(x)g(x) - f(x)g'(x)}{(g(x))^2}$.

Partial derivatives

Definition:

$$\frac{\partial f}{\partial x}(x,y) = \lim_{\Delta x \to 0} \frac{f(x+\Delta x, y) - f(x,y)}{\Delta x}.$$

$$\frac{\partial f}{\partial y}(x,y) = \lim_{\Delta y \to 0} \frac{f(x, y+\Delta y) - f(x,y)}{\Delta y}.$$

- The concept naturally extends to functions of more than two variables
- Simply hold all other variables constant and differentiate with respect to one variable
- For example: $f(x,y,\alpha,\beta) = y - \alpha - \beta x$

$$\frac{\partial f(x,y,\alpha,\beta)}{\partial \beta} = -x$$

Gradients

- For a function $f(x, y, z)$, the gradient is denoted by:

$$\nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right).$$

- It points into the direction of steepest ascent
- We will require gradients for training neural networks

Chain rule

- To determine such gradients will often require the chain rule
- **Definition:** If $y = f(u)$ and $u = g(x)$, then

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

- Examples:
  - If $f(x) = (x+2)^2$, then $f(x) = g(x)^2$ with $g(x) = x + 2$

$$\frac{df}{dx} = 2(x+2)1$$

  - If $f(x) = log(10 + x^2)$, then $f(x) = log(g(x))$ and $g(x) = 10 + x^2$

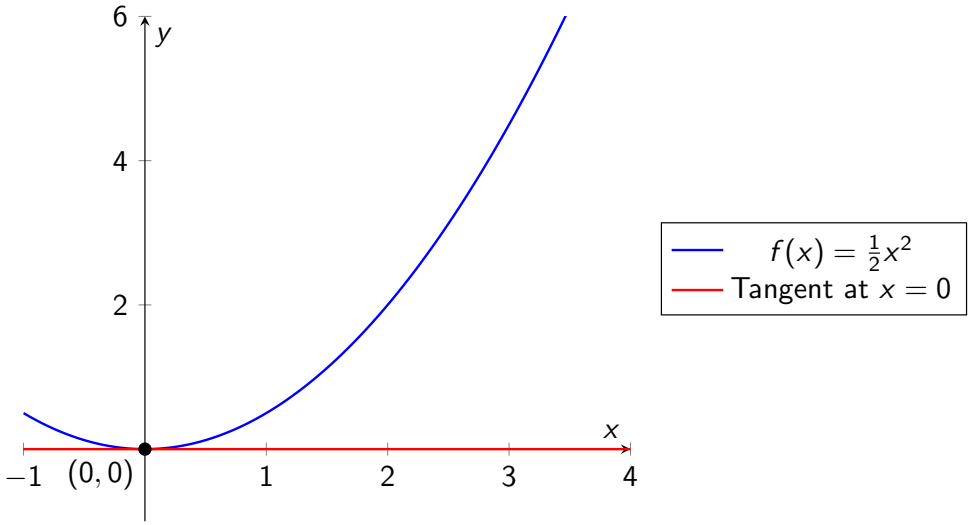
$$\frac{df}{dx} = \frac{1}{10 + x^2} 2x$$

Outline

- Vectors & matrices
- Derivatives
- Gradient descent

Optimisations with derivatives

- For differentiable functions, local minima (or maxima) occur where the derivative equals zero



The function's minimum is at $x = 0$ where its slope is zero, i.e. where $f'(x) = x = 0$

- Yet, not every point with a zero derivative is an extremum; some are saddle points where the function changes curvature

- "Learning" corresponds to moving towards a minimum in a kind of function describing the model's prediction errors, called *loss function*

Minimising loss functions

- In the following, we will define such loss functions and study how to minimise them with gradient descent
- Before discussing more complex neural networks, we will implement the approach for simple regression and classification models from scratch to gain intuition
- We will begin with linear regression, estimating the intercept and slope of the model $y_i = \alpha + \beta x_i + \varepsilon_i$ with $\varepsilon_i \sim N(0, \sigma)$

Loss of linear regression

- The goal is to minimise the simple mean squared error loss:

$$L(\alpha, \beta; \{x_i, y_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (\alpha + \beta x_i))^2$$

- For an individual observation:

$$L_i = (y_i - (\alpha + \beta x_i))^2$$

Gradient of loss

- Loss:

$$L_i = (y_i - (\alpha + \beta x_i))^2$$

- Derivatives using the chain rule:

$$\frac{\partial L_i}{\partial \alpha} = 2\left(y_i - (\alpha + \beta x_i)\right)(-1) = -2e_i$$

$$\frac{\partial L_i}{\partial \beta} = 2\left(y_i - (\alpha + \beta x_i)\right)(-x_i) = -2e_i x_i$$

where $e_i = y_i - (\alpha + \beta x_i)$

Gradient descent for linear regression

- Initialise parameters $\alpha = 0$, $\beta = 0$; choose learning rate η
- Repeat for E epochs or until convergence:
  1. Predict the residual for all observations:

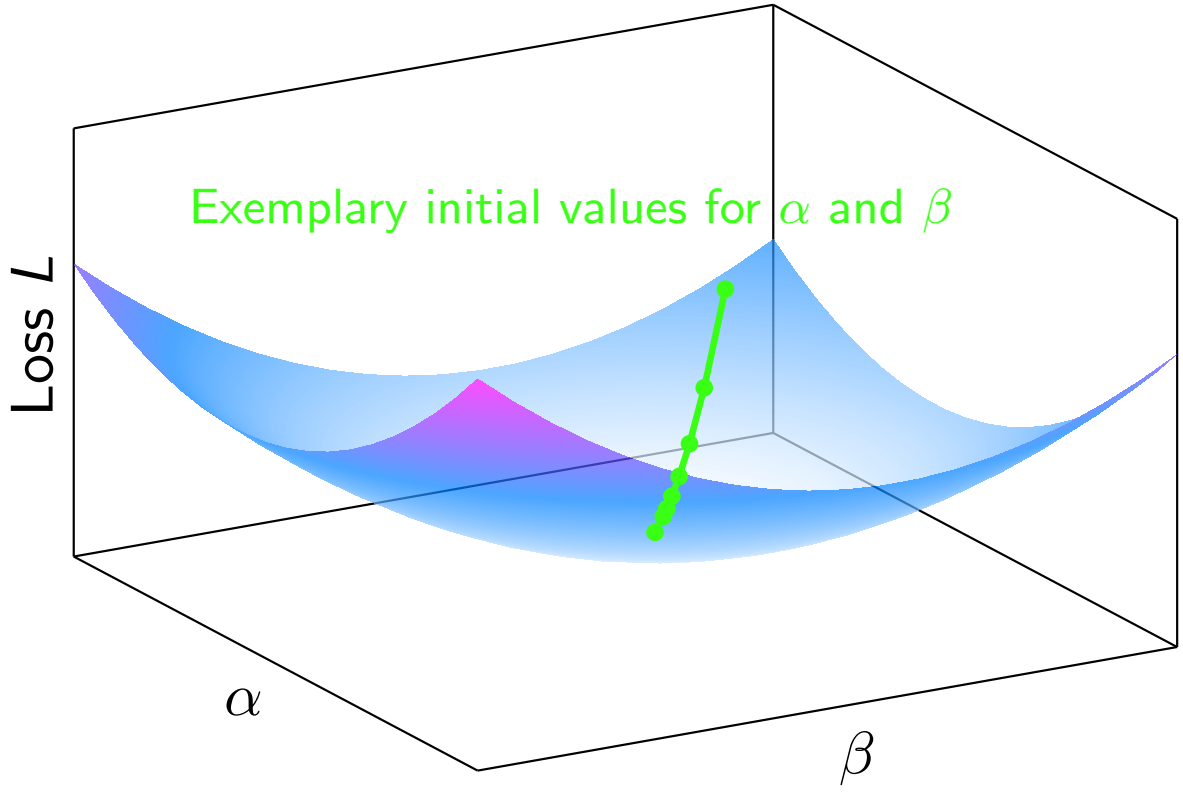
$$e_i = y_i - (\alpha + \beta x_i), \quad i = 1, \ldots, n$$

  2. Update the parameters using the average gradients:

$$\alpha \leftarrow \alpha - \eta \frac{1}{n} \sum_{i=1}^{n} (-2e_i)$$

$$\beta \leftarrow \beta - \eta \frac{1}{n} \sum_{i=1}^{n} (-2e_i x_i)$$

Visual intuition: Gradient descent



Coding

- 01-linear-regression-with-gradient-descent.ipynb

Classification with logistic regression

- The second broad class of problems in supervised machine learning is classification
- We will study the example of binary classification with logistic regression
- Conceptually, the gradient descent approach remains the same as before, we will simply minimise a different loss function
- Recall the definition of the logistic (or sigmoid) function; we will again assume a model with only one slope and an intercept

$$p_i = \frac{1}{1 + e^{-z_i}} = \frac{1}{1 + e^{-(\alpha + \beta x_i)}}$$

Loss of the logistic regression

- Logistic regression minimises the logistic (binary cross entropy) loss:

$$L(\alpha, \beta; \{x_i, y_i\}_{i=1}^n) = -\frac{1}{n}\sum_{i=1}^{n} y_i log\,(p_i) + (1 - y_i) log\,(1 - p_i)$$

- For an individual observation:

$$L_i = -\left[y_i log\,(p_i) + (1 - y_i) log\,(1 - p_i)\right]$$

- Substituting $p_i = \frac{1}{1+e^{-\alpha-\beta x_i}}$ and $1 - p_i = \frac{e^{-\alpha-\beta x_i}}{1+e^{-\alpha-\beta x_i}}$:

$$L_i = -\left[y_i log\left(\frac{1}{1+e^{-\alpha-\beta x_i}}\right) + (1 - y_i) log\left(\frac{e^{-\alpha-\beta x_i}}{1+e^{-\alpha-\beta x_i}}\right)\right]$$

- After some algebra, this simplifies to link to derivation:

$$L_i = (1 - y_i)(\alpha + \beta x_i) + log(1 + e^{-\alpha-\beta x_i})$$

Gradient of loss

- Loss: $L_i = (1 - y_i)(\alpha + \beta x_i) + log(1 + e^{-\alpha - \beta x_i})$
- Derivatives using the chain rule:

$$\frac{\partial L_i}{\partial \alpha} = (1 - y_i) + \underbrace{\frac{1}{1 + e^{-\alpha - \beta x_i}} e^{-\alpha - \beta x_i}}_{1 - p_i}(-1)$$

$$= (1 - y_i) - (1 - p_i)$$

$$= p_i - y_i$$

$$\frac{\partial L_i}{\partial \beta} = (1 - y_i)x_i + \underbrace{\frac{1}{1 + e^{-\alpha - \beta x_i}} e^{-\alpha - \beta x_i}}_{1 - p_i}(-x_i)$$

$$= (1 - y_i)x_i - (1 - p_i)x_i$$

$$= (p_i - y_i)x_i$$

Gradient descent for logistic regression

- Initialise parameters $\alpha = 0$, $\beta = 0$; choose learning rate η
- Repeat for E epochs or until convergence:

  1. Compute predicted probabilities for all observations:

$$p_i = \frac{1}{1 + \exp(-(\alpha + \beta x_i))}, \quad i = 1, \ldots, n.$$

  2. Update the parameters using the average gradients:

$$\alpha \leftarrow \alpha - \eta \, \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)$$

$$\beta \leftarrow \beta - \eta \, \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i) x_i$$

Coding

- 02-logistic-regression-with-gradient-descent.ipynb

Discussion

- This concludes the brief methods review
- The topics we revised in linear algebra, calculus and optimisation will serve as foundation for the remaining weeks of the course

Materials for further study

Excellent series on the Youtube channel *3Blue1Brown*:

- Linear algebra: `https://www.youtube.com/watch?v=fNk_zzaMoSs&list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab`
- Calculus: `https://www.youtube.com/watch?v=WUvTyaaNkzM&list=PL0-GT3co4r2wlh6UHTUeQsrf3mlS2lk6x`
- Neural networks and gradient descent: `https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi`

Derivation of the simplified logistic loss

- Loss function:

$$L_i = -\left[y_i log\left(\frac{1}{1+\exp(-\alpha-\beta x_i)}\right) + (1-y_i) log\left(\frac{\exp(-\alpha-\beta x_i)}{1+\exp(-\alpha-\beta x_i)}\right)\right]$$

- Using $log(1) = 0$, $log(a^n) = nlog(a)$, and $log(\frac{a}{b}) = log(a) - log(b)$, this simplifies to:

$$L_i = -\left[-y_i log(1+\exp(-\alpha-\beta x_i)) + (1-y_i)\left(-\alpha-\beta x_i - log(1+\exp(-\alpha-\beta x_i))\right)\right]$$

$$L_i = y_i log(1+\exp(-\alpha-\beta x_i)) + (1-y_i)\left(\alpha+\beta x_i + log(1+\exp(-\alpha-\beta x_i))\right)$$

$$L_i = (1-y_i)(\alpha+\beta x_i) + log(1+\exp(-\alpha-\beta x_i))$$

Back